PROGRAMMING IN JAVA
ASSIGNMENT 3

Student Name: M.Naga Shalini

Register Number: 192524183

Q5 – Race Conditions in a Cinema Seat-Booking System

Aim

Many customers may try to book the same seat concurrently. A race condition occurs when the final result depends on the

timing/interleaving of threads. The check-then-book sequence must be one atomic critical section.

Unsynchronized

```
public class UnsafeBooking {
    private final boolean[] booked = new boolean[10];
    private int bookingsMade = 0;
    public void book(int seat) {
        if (!booked[seat]) {
            // Makes the race easier to observe; not a correctness fix.
            try { Thread.sleep(2); }
            catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
            booked[seat] = true;
            bookingsMade++;
        }
    }
    public static void main(String[] args)
            throws InterruptedException {
        UnsafeBooking screen = new UnsafeBooking();
        Thread[] users = new Thread[100];
```

```
        for (int i = 0; i < 100; i++) {
            final int seat = i % 10;
            users[i] = new Thread(() -> screen.book(seat));
            users[i].start();
        }
        for (Thread t : users) t.join();
        System.out.println("Seats sold: " + screen.bookingsMade);
    }
}
```

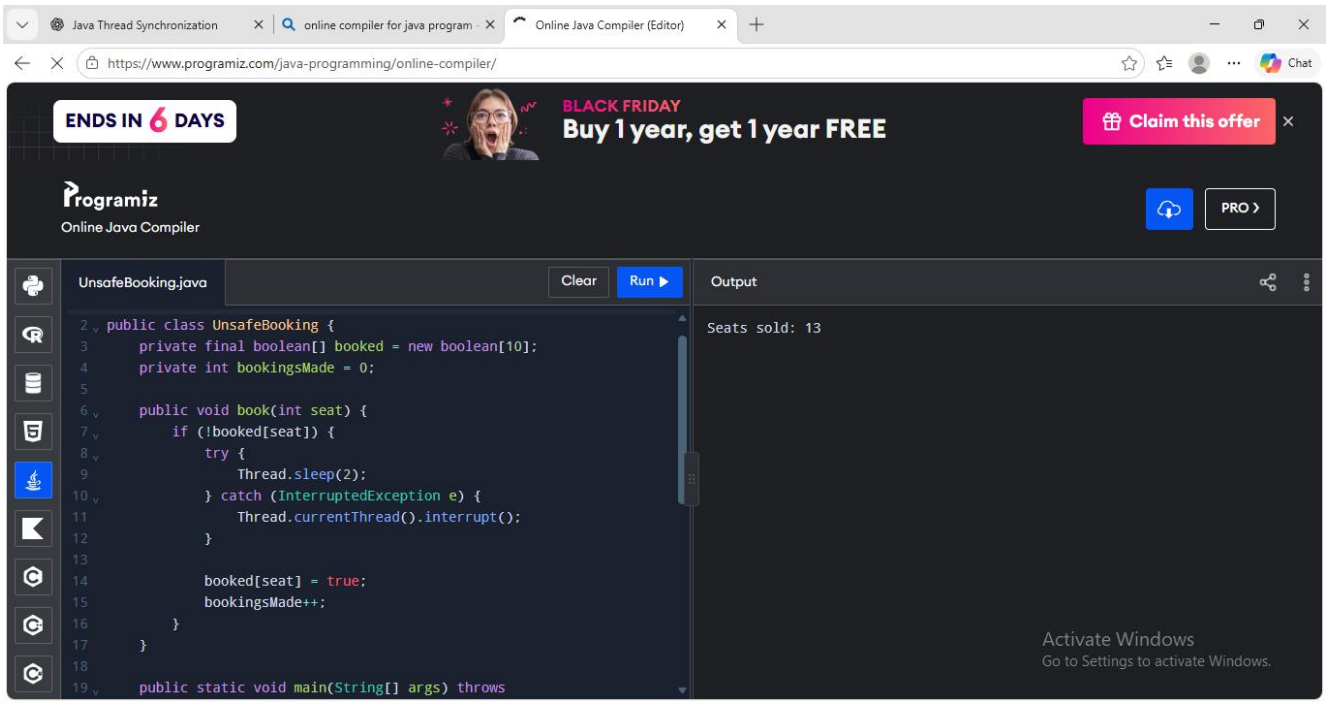
Correct synchronized method

```
public class SeatBooking {
    private final boolean[] booked;
    private int bookingsMade = 0;
    SeatBooking(int seats) { booked = new boolean[seats]; }
    public synchronized void book(int seat) {
        if (seat < 0 || seat >= booked.length) return;
        if (!booked[seat]) {
            booked[seat] = true;
            bookingsMade++;
        }
    }
    public static void main(String[] args)
            throws InterruptedException {
        SeatBooking screen = new SeatBooking(10);
        Thread[] users = new Thread[100];
        for (int i = 0; i < users.length; i++) {
            final int seat = i % 10;
            users[i] = new Thread(() -> screen.book(seat));
            users[i].start();
        }
        for (Thread t : users) t.join();
        System.out.println("Seats sold: " + screen.bookingsMade);
```

```
    }
}
```

Expected output

Seats sold: 13



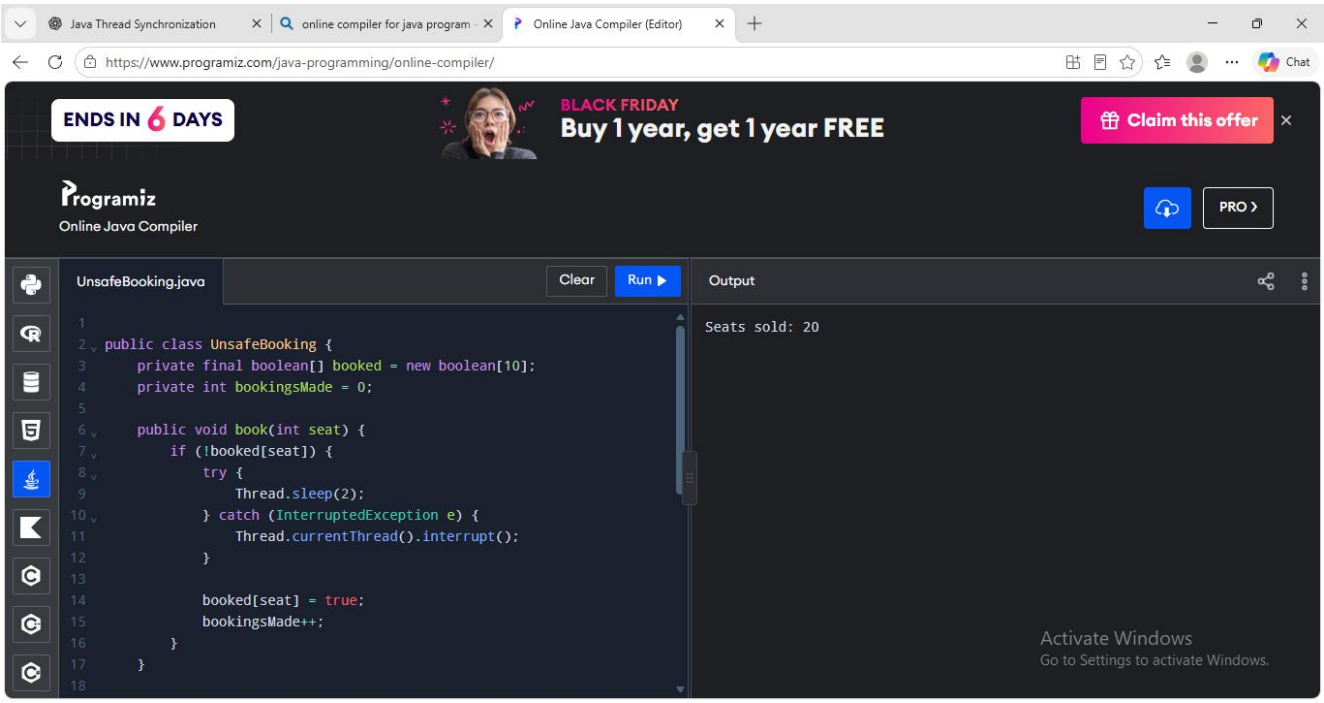
Synchronized block alternative

```
public class UnsafeBooking {
    private final boolean[] booked = new boolean[10];
    private int bookingsMade = 0;
    public void book(int seat) {
        if (!booked[seat]) {
            try {
                Thread.sleep(2);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
            booked[seat] = true;
```

```
            bookingsMade++;
        }
    }
    public static void main(String[] args) throws InterruptedException {
        UnsafeBooking screen = new UnsafeBooking();
        Thread[] users = new Thread[100];
        for (int i = 0; i < 100; i++) {
            final int seat = i % 10;
            users[i] = new Thread(() -> screen.book(seat));
            users[i].start();
        }
        for (Thread t : users) {
            t.join();
        }
        System.out.println("Seats sold: " + screen.bookingsMade);
    }
}
```

Expected output:

Seats sold: 15




```
public class UnsafeBooking {
    private final boolean[] booked = new boolean[10];
    private int bookingsMade = 0;

    public void book(int seat) {
        if (!booked[seat]) {
            try {
                Thread.sleep(2);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }

            booked[seat] = true;
            bookingsMade++;
        }
    }
```

Analysis and comparison

Two threads can both read booked[seat] as false before either sets it true. Both then increment bookingsMade. This is why the unsynchronized version may count more than ten sales. The critical section contains the availability check, state updateand counter increment. A synchronized method locks the entire method on this. A synchronized block can limit the lock toonly the critical section. Both use the same object's intrinsic monitor in the examples.

Q6 – Deadlock and Inter-Thread Communication

Part A – Wallet transfer and deadlock

```
class Wallet {
final int id;
  int balance;
Wallet(int id, int balance) {
 this.id = id;
 this.balance = balance;
```

```
    }
}
public class WalletTransfer {
    static void move(Wallet a, Wallet b, int amount) {
        Wallet first = a.id < b.id ? a : b;
        Wallet second = a.id < b.id ? b : a;
        synchronized (first) {
            synchronized (second) {
                if (amount > 0 && a.balance >= amount) {
                    a.balance -= amount;
                    b.balance += amount;
                    System.out.println("Transfer successful");
                } else {
                    System.out.println("Transfer failed");
                }
            }
        }
    }
    public static void main(String[] args) {
        Wallet w1 = new Wallet(1, 1000);
        Wallet w2 = new Wallet(2, 500);
        move(w1, w2, 200);
```

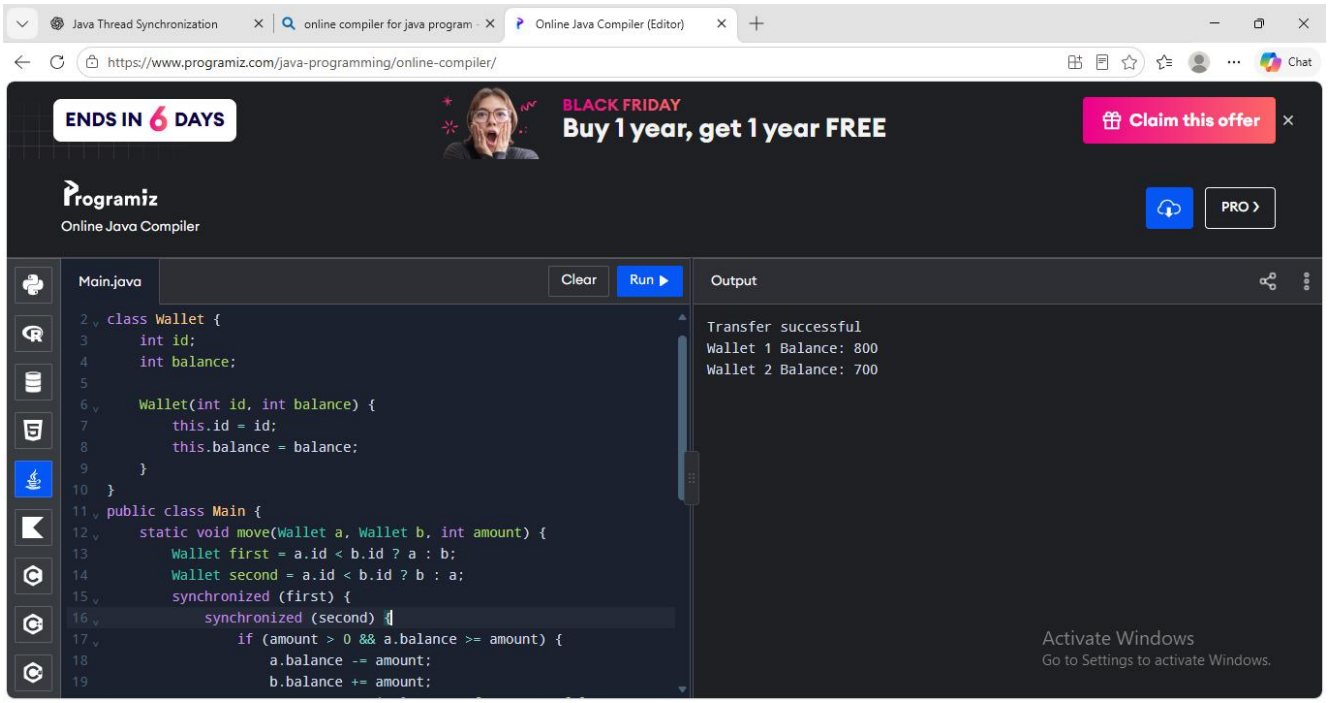
```
        System.out.println("Wallet 1 Balance: " + w1.balance);
        System.out.println("Wallet 2 Balance: " + w2.balance);
    }
}
```

Output:

Transfer successful

Wallet 1 Balance: 800

Wallet 2 Balance: 700



Part B – Producer-consumer bounded counter

```
public class Main {
private final int[] buffer = new int[5]
 private int count = 0;
 private int in = 0;
  private int out = 0;
```

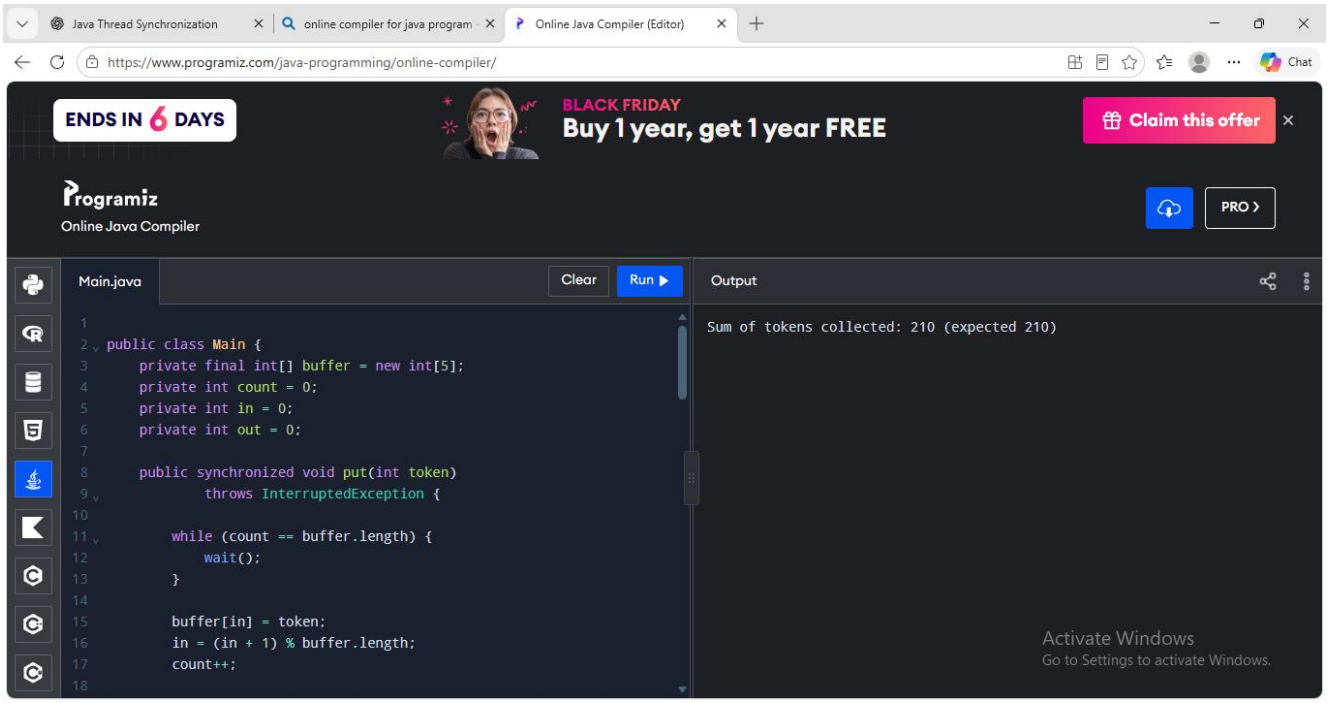
```
public synchronized void put(int token)
 throws InterruptedException {
  while (count == buffer.length) {
wait();
}
buffer[in] = token;
 in = (in + 1) % buffer.length
count++;
notifyAll();
 }
    public synchronized int take()
            throws InterruptedException {
        while (count == 0) {
            wait();
        }
 int token = buffer[out];
        out = (out + 1) % buffer.length;
        count--;
        notifyAll();
public static void main(String[] args)
            throws InterruptedException {
```

```
Main counter = new Main();
int[] collected = new int[2];
Thread cook = new Thread(() -> {
    try {
        for (int t = 1; t <= 20; t++) {
            counter.put(t);
        }
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
});
Thread[] students = new Thread[2];
for (int s = 0; s < 2; s++) {
    final int id = s;
    students[s] = new Thread(() -> {
        try {
            for (int k = 0; k < 10; k++) {
                collected[id] += counter.take();
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
```

```
            });
        }
        cook.start();
        for (Thread t : students) {
            t.start();
        }
        cook.join();
        for (Thread t : students) {
            t.join();
        }
        System.out.println("Sum of tokens collected: "
                + (collected[0] + collected[1])
                + " (expected 210)");
    }
}
```

Expected output:

Sum of tokens collected:210(expected 210)




```
public class Main {
    private final int[] buffer = new int[5];
    private int count = 0;
    private int in = 0;
    private int out = 0;

    public synchronized void put(int token)
            throws InterruptedException {

        while (count == buffer.length) {
            wait();
        }

        buffer[in] = token;
        in = (in + 1) % buffer.length;
        count++;
```

Q7 – Design a Concurrent, Fault-Tolerant Processor from First Principles

Problem interpretation and requirements

The examination cell processes 10,000 score files. Each file is independent and can be processed in parallel. A corrupt or missing file must not stop other tasks. At the end, the application must report the number of successful and failed files.

Java implementation

```
import java.nio.file.*;
import java.util.*;
import java.util.concurrent.*;
class Result {
    String file;
    boolean success;
    String message;
    Result(String file, boolean success, String message) {
```

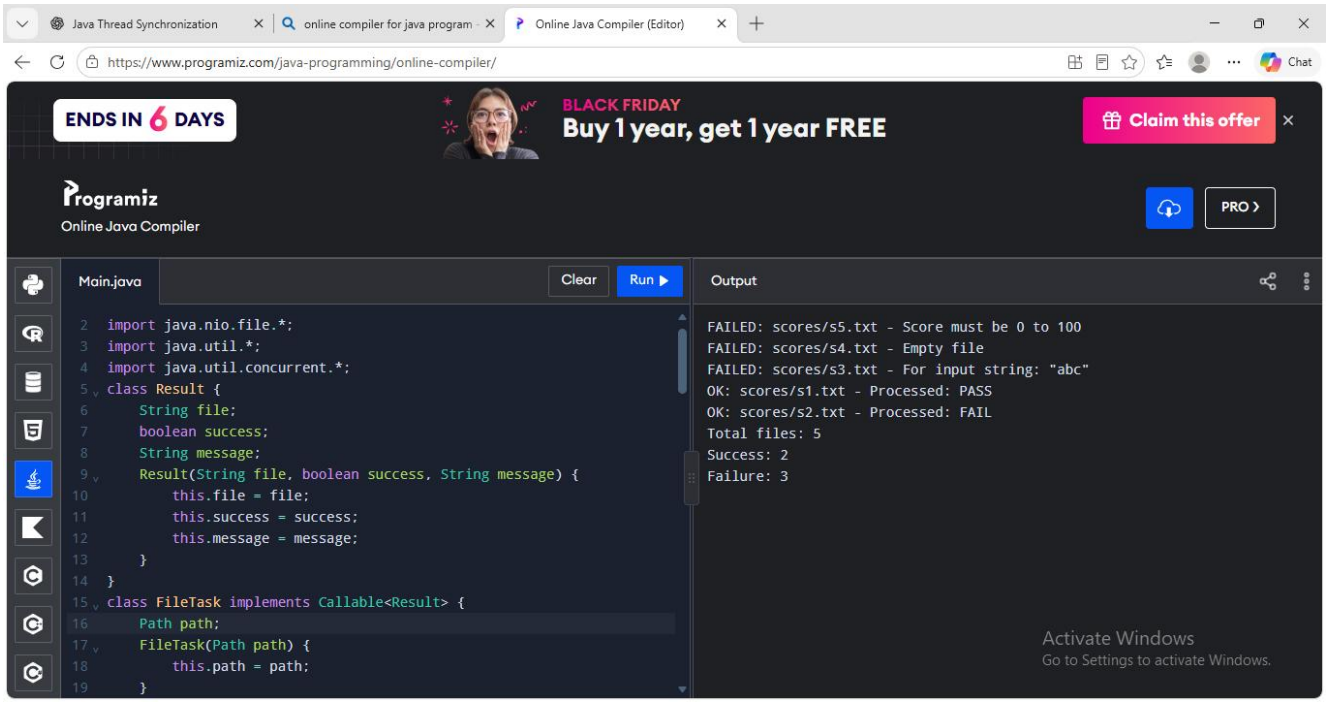
```
        this.file = file;
        this.success = success;
        this.message = message;
    }
}
class FileTask implements Callable<Result> {
    Path path;
    FileTask(Path path) {
        this.path = path;
    }
    public Result call() {
        try {
            String data = Files.readString(path).trim();
            if (data.isEmpty())
                return new Result(path.toString(), false, "Empty file");
            int score = Integer.parseInt(data);
            if (score < 0 || score > 100)
                return new Result(path.toString(), false,
                        "Score must be 0 to 100");
            String grade = score >= 50 ? "PASS" : "FAIL";
            return new Result(path.toString(), true,
                    "Processed: " + grade);
        } catch (Exception e) {
            return new Result(path.toString(), false, e.getMessage());
        }
    }
}
```

```
public class Main {
    public static void main(String[] args) throws Exception {
        Path folder = Paths.get("scores");
        Files.createDirectories(folder);
        Files.writeString(folder.resolve("s1.txt"), "85");
        Files.writeString(folder.resolve("s2.txt"), "42");
        Files.writeString(folder.resolve("s3.txt"), "abc");
        Files.writeString(folder.resolve("s4.txt"), "");
        Files.writeString(folder.resolve("s5.txt"), "150");
        List<Path> files = new ArrayList<>();
        try (DirectoryStream<Path> stream =
                Files.newDirectoryStream(folder, "*.txt")) {
            for (Path p : stream)
                files.add(p);
        }
        ExecutorService pool =
                Executors.newFixedThreadPool(4);
        List<Future<Result>> futures = new ArrayList<>();
        for (Path p : files)
            futures.add(pool.submit(new FileTask(p)));
        int success = 0;
        int failure = 0;
        for (Future<Result> future : futures) {
            Result r = future.get();
            if (r.success) {
                success++;
                System.out.println("OK: " + r.file + " - " + r.message);
```

```
            } else {
                failure++;
                System.out.println("FAILED: " + r.file + " - " + r.message);
            }
        }
        pool.shutdown();
        System.out.println("Total files: " + files.size());
        System.out.println("Success: " + success);
        System.out.println("Failure: " + failure);
    }
}
```

Expected output:

```
OK: scores/s1.txt - Processed: PASS
OK: scores/s2.txt - Processed: FAIL
FAILED: scores/s3.txt - For input string: "abc"
FAILED: scores/s4.txt - Empty file
FAILED: scores/s5.txt - Score must be 0 to 100
Total files: 5
Success: 2
Failure: 3
```



Final conclusion

The assignment demonstrates exception handling, meaningful custom exceptions, safe resource management, thread creation, synchronization, deadlock prevention, inter-thread communication and concurrent task processing.